

Instructor: Date:

Learner: Regional Branch:

Program: Center:

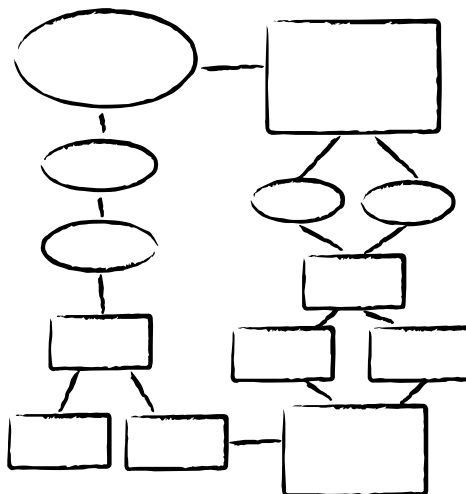
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Following steps



Learning Outcome:

Identify information about algorithms related to software programming in simple spoken and written situations.

Ask your partner:

- Do you accomplish procedures to get a good result in your life? Why? Why not?
- Do you think is important to write the steps to get the result you want? Why? Why not?

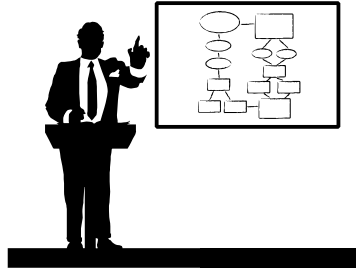
Can Do Statements:

- I can say what a programming algorithm is.
- I can discuss a simple example of an algorithm in a numbered list.
- I can talk about a simple example of an algorithm in a flow chart.



Let's Explore!

A Lesson on Programming Algorithms



Instructions: Read the text below.

I always get questions about programming algorithms, so let's start this session by giving a fast but precise explanation on this topic. I do this because most of the people come to my class thinking this is a complicated topic, but believe me when I say programming algorithms is not difficult.

An algorithm is like a numbered list of things you need to do to get a result. A programming algorithm is the same, but the procedure is described by using a programming language, which is the one the computer can understand. The computer receives the input that is the information needed (the steps) to develop the procedure. Then, when the computer does the procedure, an outcome is expected; the outcome is the final result of the procedure. Let's imagine you are cooking using the help of a recipe. After following each step, the outcome is the finished dish.

When doing this procedure all the steps have to be very clear; there is no room for interpretation when writing algorithms. You can't make a formula to bake an apple pie and as a result, have a chocolate cake. When something like this happens, it is because the instructions are not precise, they are ambiguous, and that can't happen on programming algorithms.

As I said before, an algorithm is a list, but you can also do it as a flowchart. Just in case, let's have a little review on what that is. In this case, when writing the algorithm you need the same steps just that you use shapes and arrows, creating a map route to the desired result. In other words, with a flowchart, you can visualize the route you are taking to solve a problem on your computer.

Let's see an example of a flowchart for an email password. This kind of flowchart is made of several symbols that have specific purposes. The Terminal or Terminator, which indicates where the process starts or finishes. The rectangle, which represents the Process Object and usually contains a verb that specifies the action to be taken; in the case of the email password, it usually contains the verb *change* or even the words *error message*, in case the user makes a mistake when entering the information. We also have the Data Object or I/O shape, a parallelogram which shows the inputs and outputs from a process; for the email password, it contains the words *+Old password*, *+New password*, *+Verify password*. Finally, there is the Decision/Conditional Object, which is represented as a diamond with a question or statement in it and two arrows coming out of it representing its two possible answers, yes or no; for our example, these diamonds contain the sentences *"Load data to form"*, *"Is validation data?"* and *"Update password to database"*.





Let's Practice!

Algorithm Sequence and Organization

Instructions: Match column A with column B.

COLUMN A

1. output
2. Input
3. flow chart
4. Numbered list

COLUMN B

- a. Let me tell you that this is a set of steps designed to present an algorithm.
- b. Let me tell you that expected results after data processing.
- c. Let me explain you that here the information is entered into a system to be analyzed.
- d. Let me explain you that this diagram represents operations sequentially.



Let's Practice!

Do you know how a flow chart can support a programming algorithm?

Instructions: Select the correct answer.

1. You _____ a formula to make an apple pie and have a chocolate cake as a result.
 - a. can't make
 - b. can make
 - c. makes
 - d. couldn't make
2. Sometimes language can be ambiguous and that _____ when programming algorithms.
 - a. can happen
 - b. could has
 - c. can fulfill
 - d. can't happen
3. You _____ shapes and arrows, creating a map route to the desired result.
 - a. can use
 - b. can't use
 - c. allow use
 - d. uses
4. With a flowchart you _____ the route you are taking to solve a problem in your computer.
 - a. can't visualize
 - b. couldn't visualize
 - c. can visualize
 - d. visualizing
5. The procedure _____ created by using a flowchart and programming language.
 - a. can't be
 - b. can be
 - c. couldn't be
 - d. been





Extension Activity

More on Programming Algorithms

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for information about more programming algorithm samples.
- Write notes and select one of them.
- Design a poster to describe it, use visual aids.
- Share with your partners and solve questions.



Target Words

- **parallelogram** (noun): a quadrilateral whose opposite sides are parallel and equal in length
e.g. We also have the Data Object or I/O shape, a parallelogram which shows the inputs and outputs from a process; for the email password it contains the words Old password, New password, and Verify password.
- **arrow** (noun): line with a wedge-shaped end that shows the direction of something
e.g. Finally, there is the Decision/Conditional Object, which is represented as a diamond with a question or statement in it and two arrows coming out of it representing its two possible answers, yes or no.
- **shapes** (noun): an external specific outline or figure
e.g. In this case, when writing the algorithm you need the same steps just that you use shapes and arrows, creating a map route to the desired result.

Self-Assessment

Yes

No

- I look for information about more programming algorithm samples.
- I describe a programming algorithm sample.
- I talk about programming algorithm and its features.

☐☐☐☐☐☐



Learning Points

Simple Present With Verb to Be Negative in Statements

The verb **to be** is one of the most important verbs in English, and it is also one of the easiest to learn.

You use the verb to be to express the general status and characteristics of a person, an animal or an object.

In this case, you can use the verb to be in negative form to say that something is not a general state or is not true at the moment of speaking. Let's see the next structure.

Subject + verb to be + not + compliment.

Example:

- *Programming algorithms **are not** a computer code.*

You can also use the contractions to express a negative sentence. Let's see the next chart.

Verb to be contraction with the subject	Verb to be contraction with the "not"
I'm not	I'm not
You're not	You aren't
He's not/ she's not/ it's not	He isn't/ she isn't/ it isn't
We're not	We aren't
They're not	They aren't

Examples

- *But believe me when I say programming algorithms are not difficult*
- *All the steps have to be very clear; there is not space for interpretation when writing algorithms.*





Learning Points

Algorithm Flow Chart

You can write **programming algorithms** in two ways, as a numbered list or as a flowchart.

Using a flowchart to write a programming algorithm is very simple. All you have to do is to organize the steps of what you are going to do, as a recipe. The only difference of using a flowchart is that we need to use shapes and arrows to connect the steps among them.

Let's imagine we are going to create a password for our email account.

As a list it would look like this:

1. Start.
2. Create a variable to receive the user's password.
3. Clear the variable in case it is not empty.
4. Ask the user for a password.
5. Store the response in the variable.
6. Check if the stored response is a valid password.
7. Not valid? Go back to step 3.
8. End.

This same exercise as a flowchart look like this:

